

The use of copyrighted materials in all formats, including the creation, online delivery, and use of digital copies of copyrighted materials, must be in compliance with U.S. Copyright Law (<http://www.copyright.gov/title17/>). Materials may not be reproduced in any form without permission from the publisher, except as permitted under U.S. copyright law. **Copyrighted works are provided under Fair Use Guidelines only to serve personal study, scholarship, research, or teaching needs.**

Maurizio Gabbrielli and Simone Martini

Programming Languages: Principles and Paradigms

 Springer

Prof. Dr. Maurizio Gabbrielli
Università di Bologna
Bologna
Italy

Prof. Dr. Simone Martini
Università di Bologna
Bologna
Italy

Series editor
Ian Mackie

Advisory board

Samson Abramsky, University of Oxford, UK
Chris Hankin, Imperial College London, UK
Dexter Kozen, Cornell University, USA
Andrew Pitts, University of Cambridge, UK
Hanne Riis Nielson, Technical University of Denmark, Denmark
Steven Skiena, Stony Brook University, USA
Iain Stewart, University of Durham, UK
David Zhang, The Hong Kong Polytechnic University, Hong Kong

ISSN 1863-7310

ISBN 978-1-84882-913-8

e-ISBN 978-1-84882-914-5

DOI 10.1007/978-1-84882-914-5

Springer London Dordrecht Heidelberg New York

British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

Library of Congress Control Number: 2010922995

Translated from the original Italian edition: "Linguaggi di programmazione: principi e paradigmi", published in 2006 by McGraw-Hill Companies, Publishing Group Italia.

© Springer-Verlag London Limited 2010

Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Designs and Patents Act 1988, this publication may only be reproduced, stored or transmitted, in any form or by any means, with the prior permission in writing of the publishers, or in the case of reprographic reproduction in accordance with the terms of licenses issued by the Copyright Licensing Agency. Enquiries concerning reproduction outside those terms should be sent to the publishers.

The use of registered names, trademarks, etc., in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant laws and regulations and therefore free for general use.

The publisher makes no representation, express or implied, with regard to the accuracy of the information contained in this book and cannot accept any legal responsibility or liability for any errors or omissions that may be made.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Chapter 11

The Functional Paradigm

In this chapter, we present the main properties of functional programming. In functional programming, computation proceeds by rewriting functions and not by modifying the state. The fundamental characteristic of the languages in this paradigm, at least in their “pure” form, is precisely that of not possessing the concept of memory (and therefore side effect). Once an environment is fixed, an expression always denotes the same value.

We will discuss the pure paradigm in the first sections, explaining the fundamental aspects. Functional programming languages, however, merge these “pure” ingredients in a context that adds many other mechanisms; we will review them in Sect. 11.3.

We will touch on the SECD machine, an abstract machine for higher-order functional languages which constitutes the prototype of many real implementations.

We will, at this point, be in a position to discuss the reasons why the functional programming paradigm is interesting with respect to ordinary imperative languages.

The chapter concludes with a more theoretical section which provides a succinct introduction to the λ -calculus, a formal system for computability which inspires all functional languages and which has, since the time of ALGOL and LISP, been a constant model for the design of programming languages.

11.1 Computations without State

Though developed and abstract, all conventional languages base their computational model on the transformation of the *state*. The heart of this model is the concept of *modifiable variable*, that is, a container with a name to which, during the computation, can be assigned different values, while the same association is always maintained in the environment. Correspondingly, the principal construct in conventional languages is *assignment*, which modifies the value contained in a variable (but does not modify the association between the name of the variable and the location to which it corresponds; it modifies the r-value but not the l-value, which is fixed once and for all when the variable is declared).

Conventional languages differ in their level of abstraction, their types, in the constructs which allow the manipulation of variables, but they all share the same computational model. This is an abstract view of the underlying, conventional physical machine. Computation proceeds by modifying values stored in locations. This is an extremely important model. It is called the *von Neumann Machine*, named after the Hungarian-American mathematician who, in the 1940s, saw that the Turing Machine (see box on page 61) could be engineered into a physical prototype, thereby originating the modern computer.

This is not, however, the only possible model upon which to base a programming language. It is possible to compute without using modifiable variables, that is without referring to the concept of state. The computation proceeds not by modifying the state but by *rewriting* expressions, that is by changes that take place only in the environment and do not involve the concept of memory. If there are no modifiable variables, there is no longer a need for assignment, that is the principal command in conventional languages. The entire computation will be expressed in terms of the sophisticated modification of the environment in which the possibility of manipulating (higher-order, see Sect. 7.2) functions plays a fundamental role.

Without assignment, iteration also loses its real sense. A loop can only repeatedly modify the state until the values of certain variables satisfy a guard. The reader already knows (see Chap. 6) that iterative and recursive constructs are two mechanisms permitting infinitely long (and possibly divergent) computations. In the stateless computational model, iteration disappears, recursion remains and becomes the fundamental construct for sequence control.

Higher-order functions and recursion are the basic ingredients of this stateless computational model. The programming languages which presuppose this model are called *functional languages* and the paradigm that results from this is called the *functional programming paradigm*.

This is a paradigm as old as the imperative one. Since the 1930s, beside the Turing Machine, there has existed the λ -calculus, an abstract model for characterising computable functions that is based on exactly the concept we have been briefly explaining. LISP was the first programming language explicitly inspired by the λ -calculus and many others have followed in the intervening years (Scheme, ML, in all its different dialects, Miranda, Haskell, to name only the most common). Among these, only Miranda and Haskell are “purely functional”; the others also have imperative components (often to assist programmers accustomed to conventional languages) but the structure of these languages assigns a subordinate role to them.

In this chapter, we undertake an introduction to the pure functional paradigm; it is not a superficial one and we will discuss different general aspects of the paradigm. To make the treatment more concrete, it is desirable to refer to a specific language rather than to use a neutral pseudocode. ML has been chosen for this because, through its coherence and elegance of design, it is the most suited of them all to didactic presentation. It is not our aim to introduce the language (for which, the reader should refer to the bibliography), but only to use the syntax in an instrumental fashion to discuss some general questions that apply also to other functional languages.

11.1.1 Expressions and Functions

In the usual mathematical practice, there is some ambiguity about when we are defining a function and when we are applying it to a value. It is not uncommon to encounter expressions of the kind:

Let $f(x) = x^2$ be the function that associates with x its square. If now we have $x = 2$, it follows that $f(x) = 4$.

The syntactic expression $f(x)$ is used to denote two things that are quite different: the first time, it serves to introduce the *name*, f , of a specific function; the second time, it serves to denote the result of *applying* the function f to a specified value. In mathematical practice, this ambiguity is completely innocuous because the context helps us to distinguish which of the two uses is intended. The same does not hold for an artificial language (such as a programming language) which describes functions. Here, it is appropriate to distinguish the two cases.

When a mathematician asserts that they are defining the function, $f(x)$, in reality, they are defining the function f with one *formal* parameter, x , which serves to indicate the transformation that f applies to its argument. To distinguish linguistically between the name and the “body” of the function, following ML syntax, we can write:

```
val f = fn x => x*x;
```

The reserved word `val` introduces a declaration. The declaration is used to extend the environment with a new association between a name and a value. In our case, the name f is bound to the transformation of x into $x * x$. In all functional languages, functions are *expressible* values; that is, they can be the result of the evaluation of a complex expression. In our case, the expression on the right of the `=` and introduced by `fn` is an expression that denotes a function.

For the application of a function to an argument, we retain the traditional notation, writing $f(2)$ or $(f\ 2)$, or $f\ 2$, for the expression that results from the application of a function f to the argument 2. We can also use `val` to introduce new names, as in:

```
val four = f 2;
```

The introduction of specific syntax for an expression which denotes a function has an important consequence. It is possible to write (and possibly apply) a function without having necessarily to assign it a name. For example, the expression

```
(fn y => y+1) (6);
```

has the value 7 which results from the application of the (anonymous) function `fn y => y+1` to the argument 6. To make the notation less imposing, we assume (as does ML) that application can be denoted by simple juxtaposition (that is without parentheses) and that it associates to the left (prefix notation). If g is the name of a function,

`g a1 a2 ... ak`

will mean:

`(...((g a1) a2) ... ak).`

Nothing prevents one functional expression from appearing inside another, as in

```
val add = fn x => (fn y => y+x);
```

The value `add` is a function which, given an argument, `x`, returns an anonymous function which, given an argument `y`, returns `x+y`. We can use `add` in many different ways:

```
val three = add 1 2;
val addtwo = add 2;
val five = addtwo 3;
```

Note that, in particular, `addtwo` is a function which is obtained as the result of the evaluation of another expression.

The notation which uses `val` and `fn` is important but is a little verbose. ML also allows the use of a more compact notation which resembles the usual way of defining functions in a programming language. The first function, `f`, which we defined (the one which computes the square of its argument) could also be defined as follows:

```
fun f x = x*x;
```

In general, a definition of the form:

```
fun F x1 x2 ... xn = body;
```

is only syntactic sugar (that is, in programming language jargon, only a nicer abbreviation) for:

```
val F = fn x1 => (fn x2 => ... (fn xn => body) ...);
```

As a last example of functions which manipulate functions, this time in the form of a formal parameter, consider the definition:

```
fun comp f g x = f(g(x));
```

This returns the function composed of its first two arguments (which are, in their turn, functions).

Finally, every functional program permits the definition of recursive functions. Assuming that we have available a *conditional expression* (which in ML is written with the familiar `if then else` syntax), we can define the usual factorial as:

```
fun fatt n = if n=0 then 1 else n*fatt(n-1);
```

11.1.2 Computation as Reduction

If we exclude arithmetic functions (which we can assume predefined with the usual semantics) and the conditional expression, at the conceptual level, we can describe the procedure used to transform a complex expression into its value (*evaluation*) as the process of *rewriting*. We call this process *reduction*. In a complex expression, a subexpression of the form “function applied to an argument” is textually replaced by the body of the function in which the formal parameter is replaced, in its turn, by the actual parameter.¹ We can compute a simple expression using this computational model (we use $\rightarrow$ to indicate a reduction step).

```
fact 3  $\rightarrow$  (fn n => if n=0 then 1 else n*fact(n-1)) 3
 $\rightarrow$  if 3=0 then 1 else 3*fact(3-1)
 $\rightarrow$  3*fact(3-1)
 $\rightarrow$  3*fact(2)
 $\rightarrow$  3*((fn n => if n=0 then 1 else n*fact(n-1)) 2)
 $\rightarrow$  3*(if 2=0 then 1 else 2*fact(2-1))
 $\rightarrow$  3*(2*fact(2-1))
 $\rightarrow$  3*(2*fact(1))
 $\rightarrow$  3*(2*((fn n => if n=0 then 1 else n*fact(n-1)) 1))
 $\rightarrow$  3*(2*(if 1=0 then 1 else 1*fact(1-1)))
 $\rightarrow$  3*(2*(1*fact(0)))
 $\rightarrow$  3*(2*(1*((fn n => if n=0 then 1 else n*fact(n-1)) 1)))
 $\rightarrow$  3*(2*(1*(if 0=0 then 1 else n*fact(n-1))))
 $\rightarrow$  3*(2*(1*1))
 $\rightarrow$  6
```

Note how, with the exception of arithmetic calculations and the use of the conditional expression, all of the rest of the computation proceeds by symbolic manipulation of strings: no variables, no update of stacked variables. Figure 11.1 contains another example of pure symbolic manipulation. (It is a complicated way to write the identity function!) (While studying the figure, recall that `fun` is just an abbreviation for `val ... fn.`)

Finally, the reader will have no difficulty in convincing themselves that, given the definition

```
fun r x = r(r(x));
```

each computation which involves an evaluation of r resolves into an infinite rewriting. We say, in such a case, that the computation *diverges* and that the result is undefined.

¹The reader will certainly have recognised in this description the “copy rule” we already discussed in the context of the by-name parameter passing. For the time being, we will be vague about the exact semantics to assign to this process—all of Sect. 11.2 will be devoted to it.

```

fun K x y = x;
fun S p q r = p r (q r);
val a = ...;

S K K a → (fn p => (fn q => (fn r => p r (q r)))) K K a
        → (fn q => (fn r => K r (q r))) K a
        → (fn r => K r (K r)) a
        → K a (K a)
        → (fn x => (fn y => x)) a (K a)
        → (fn y => a) (K a)
        → a

```

Fig. 11.1 Some definitions and a computation using rewriting

11.1.3 The Fundamental Ingredients

In these first subsections, we have introduced all the fundamental ingredients of the pure functional paradigm. We can make this precise and summarise the main concepts in the following way.

From the syntactic viewpoint, a language such as the one under consideration has no commands (there being no state to modify using side effects) but only expressions. Apart from possible values and primitive operators for data (such as integer, boolean, characters, etc.) and the conditional expression, the two principal constructs for defining expressions are:

- *Abstraction* which, given any expression, `exp` and an identifier, `x`, allows the construction of an expression `fn x => exp` denoting the function that transforms the formal parameter `x` into `exp` (the expression `exp` is “abstracted” from the specific value bound to `x`).
- The *application* of an expression, `f_exp`, to another expression, `a_exp`, which we write `f_exp a_exp`, which denotes the application of the function (denoted by) `f_exp` to the argument (denoted by) `a_exp`.

There are no constraints on the possibilities of passing functions as arguments to other functions, or to returning functions as the results of other (higher-order) functions. As a consequence, there is perfect *homogeneity* between programs and data.

From the semantic viewpoint, a program consists of a series of value definitions, each of which inserts a new association into the environment and can require the evaluation of arbitrarily complex expressions. The presence of higher-order functions and the possibility of defining recursive functions makes this definition mechanism flexible and powerful.

To a first approximation, the semantics of computation (*evaluation*) refers to no linguistic aspects other than the ones introduced so far. It can be defined using simple symbolic rewriting of strings (*reduction*), which repeatedly uses two main operations to simplify expressions until they reach a simple form which immediately denotes a value. The first of these operations is the simple search through the environment. When an identifier is determined as being bound in an environment,

replace the identifier by its definition. For example, in Fig. 11.1, we used this operation for the first and the fifth reduction. In what follows this step will not be explicitly considered again. We will consider a name as a simple abbreviation for the value associated with it.

The second operation, which is more interesting, deals with a functional expression applied to an argument and uses a version of the copy rule (which in this context is called the β -rule).

Definition 11.1

Redex A *redex* (which stands for a *reducible expression*) is an application of the form $((\text{fn } x \Rightarrow \text{body}) \text{ arg})$.

Reductum The *reductum* of a redex $((\text{fn } x \Rightarrow \text{body}) \text{ arg})$ is the expression which is obtained by replacing in *body* each (free) occurrence of the formal parameter, x , by a copy of *arg* (avoiding variable capture², see p. 175).

β -rule An expression, *exp*, in which a redex appears as a subexpression is reduced (or rewrites, simplifies) to *exp1* (notation: $\text{exp} \rightarrow \text{exp1}$), where *exp1* is obtained from *exp* by replacing the redex by its reductum.

From an implementation viewpoint, finally, every abstract machine for functional languages adopts extensive garbage-collection mechanisms because, in the case in which functions are returned as values by other functions, we know that the local environment must be preserved for an unlimited period of time (see Sect. 7.2.2).

If these are the cardinal notions of a functional language, on a more attentive reading, these few lines raise more problems than they solve. We undertake a deeper analysis of these semantic concepts in the next section.

11.2 Evaluation

In the brief semantic description with which we concluded the last section, we did not provide details about two fundamental aspects:

- What is the termination condition for reduction (that is what does “a simple form which immediately denotes a value” mean?)
- What precise semantics must be given to the β -rule, not just concerning possible variable capture (which we already described in the context of parameter passing) but more importantly, the order to follow during rewriting should more than one redex be present in the same expression.

²In the context of functional programming, identifiers bound to values are often referred to as “variables”. The reader is by now experienced enough to allow us to continue with this traditional use without being confused by the absence of modifiable variables in pure functional programming languages.

11.2.1 Values

A *value* is an expression which cannot be further rewritten. In a functional language, there are values of two kinds: values of primitive type and functions. There is little to say about values of simple types. If the language provides some primitive types (integers, booleans, characters, etc.) is clear that a set of primitive values is associated with each type thus defined and that these values do not admit evaluation (for example, constants of type integer, boolean, characters, etc.). In the example of the previous subsection, we terminated the evaluation of `fact` when we reached primitive values of integer type: 3, 2, 1.

Functional values are more interesting. Let us consider the following definition:

```
val G = fn x => ((fn y => y+1) 2);
```

We have said that a definition entails the evaluation of the expression on the right of the equality and the binding of the value thus derived to the name on the left of the `=`. But, in this case, it is not immediately clear what the value to be associated with `G` is. Do we have:

```
fn x => 3
```

in which we have rewritten the body of `G` by evaluating the redex it contains, or do we have

```
fn x => ((fn y => y+1) 2)
```

in which there has been no evaluation in the body of `G`? The first case appears to be the one that more respects the informal semantics that we gave at the end of the last subsection. The second, on the other hand, is closer to the usual definition of function in a conventional language, in which the body of a function is only evaluated when it is called.

Although at first sight, this can appear strange, it is the second of the above that is adopted for all functional languages in common use. Evaluation does not occur “under” an abstraction. Every expression of the form

```
fn x => exp
```

represents a value, so redexes possibly contained in `exp` are *never* rewritten until the expression is applied to some argument.

11.2.2 Capture-Free Substitution

To implement capture-free substitution, we saw in Chap. 7 that closures can be used. This is, in effect, the mechanism also used by abstract machines for functional

Fig. 11.2 An expression with more than one redex

```

fun K x y = x;
fun r z = r(r(z));
fun D u = if u=0 then 1 else u;
fun succ v = v+1;

val v = K (D (succ 0)) (r 2);

```

languages (see Sect. 11.4). For the elementary description we are giving at present, though, a syntactic convention suffices. In every expression, there are never two formal parameters with the same name and the names of the possible variables that are not formal parameters are all distinct from those of the formal parameters.

Using this convention, there will never be variable capture in the simple examples that we consider.³

11.2.3 Evaluation Strategies

In Chap. 6, while discussing expressions, we saw how every language must fix a specific strategy (that is a fixed order) for the evaluation of expressions. The presence of higher-order functions in functional languages makes this question even more fundamental. To clarify the problem, consider the definitions in Fig. 11.2. Which value is associated with v and how is it determined?

The β -rule on its own is not of much use because, in the right-hand part of the definition of v , there are 4 redexes (after the expansion of the names with the values associated with them by the definitions):

```

K (D (succ 0))
D (succ 0)
succ 0
r 2

```

Which of them is reduced first? Every commonly used language uses a leftmost strategy which reduces the redexes starting with the one at the leftmost end. But even having stipulated this, it is not clear which is the leftmost redex of

```

K (D (succ 0))
D (succ 0)
succ 0

```

because these three redexes are superimposed on each other. Having fixed a leftmost evaluation order, we then find that we have three different strategies.

³In the more general case, if we do not want to bring into play the concept of closure, for the correct description of computation using rewriting, it is necessary precisely to define the concepts of bound and free variable and substitution, all concepts which we treat formally in Sect. 11.6.

Evaluation by value In evaluation by value (which is also called applicative-order evaluation or *eager* evaluation, or *innermost* evaluation), a redex is evaluated only if the expression which constitutes its argument part is already a value.

More precisely, leftmost evaluation in applicative order works as follows.

1. Scan the expression to be evaluated from the left, choosing the first application encountered. Let it be $(f_exp\ a_exp)$.
2. First evaluate (recursively applying this method) f_exp until it has been reduced to a value (of functional type) of the form $(fn\ x\ =>\ \dots)$.
3. Then evaluate the argument part, a_exp , of the application, so that it is reduced to a value, val .
4. Finally, reduce the redex $((fn\ x\ =>\ \dots)\ val)$ and goto to (1).

Considering Fig. 11.2, case (1) first chooses the application $K\ (D\ (succ\ 0))$. Now some elementary applications of (1), (2) and (3) will serve to show that K , D and $succ$ are already values (recall that we are implying that a name is an abbreviation for the expression associated with it). The first redex to be reduced is then $succ\ 0$ (that is, $((fn\ v\ =>\ v + 1)\ 0)$) which will be completely evaluated and yields 1. Then the redex $(D\ 1)$ (that is $((fn\ u\ =>\ if\ u = 0\ then\ 1\ else\ u)\ 1)$) is reduced to give the value 1. Then $(K\ 1)$ is evaluated to produce the value $(fun\ y\ ==>\ 1)$. At this point in the evaluation, the expression has become

```
(fun y => 1) (r 2)
```

Since the functional part of this application is already a value, the strategy prescribes that the argument $(r\ 2)$ is evaluated. The evaluation leads to rewriting to $r\ (r\ 2)$, then to $r\ (r\ (r\ 2))$ and so on in a divergent computation.

No value is therefore associated with v because the computation diverges.

Evaluation by name In the evaluation by name strategy (which is also called *normal order* or *outermost*), a redex is evaluated before its argument part.

More precisely, leftmost evaluation in normal order proceeds as follows.

1. Scan the expression to be evaluated from the left, choosing the first application encountered. Let it be $(f_exp\ a_exp)$.
2. First evaluate f_exp (recursively applying this method) until it has been reduced to a value (of functional type) of the form $(fn\ x\ =>\ \dots)$.
3. Reduce the redex $((fn\ x\ =>\ \dots)\ a_exp)$ using the β -rule and goto (1).

Considering Fig. 11.2, the first redex to be reduced is therefore:

```
K (D (succ 0))
```

It is rewritten to:

```
fn y => D (succ 0)
```

which is a functional value. The expression now is of the form:

```
(fn y => D (succ 0)) (r 2)
```

for which the strategy prescribes reducing the outermost redex, so we obtain:

```
D (succ 0)
```

Now reducing this expression, we obtain:

```
if (succ 0)=0 then 1 else (succ 0)
```

and then:

```
if 1=0 then 1 else (succ 0)
```

and finally

```
succ 0
```

from which we obtain the final value, 1, which is then associated with v .

Lazy evaluation In evaluation by name, a single redex might have to be evaluated more than once because of some duplication that has occurred during rewriting. In the example that we are discussing, the redex `(succ 0)` is duplicated because of the function `D` and is reduced twice⁴ in the conditional expression that forms the body of `D`. This is the price that must be paid for postponing the evaluation of an argument until after the application of a function (and it is really this that allows evaluation by name to obtain a value where evaluation by value would diverge). But it is very expensive in terms of efficiency (when the duplicated redex requires a significant amount of computation).

To obviate this problem and maintain the advantages of evaluation by name, the lazy strategy proceeds like that by name but the first time that a “copy” of a redex is encountered, its value is saved and will be used should any other copies of the same redex be encountered.

The by-name and lazy strategies are examples of the *call by need* strategies in which a redex is reduced only if it is required by the computation.

11.2.4 Comparison of the Strategies

In the last subsection, the by-name strategy produces a value when the by-value strategy diverges. It is sensible to ask the question as to whether it can be the case that the two strategies produce *distinct* values for the same expression.

⁴The purist does not speak of copies of redexes but will rather say that the redex `(succ 0)` has given way to two *residuals*, each of which has been reduced independently of the other.

An answer to this question is given by the following theorem which expresses one of the most important characteristics of the *pure* functional paradigm. We say that an expression in the language is *closed* if all of its variables are bound by some *fn*. Let us recall, then, from Sect. 11.2.1, that by *primitive value*, we mean a value of a primitive type (integer, boolean, character, etc.), excluding, therefore, functional values.

Theorem 11.2 *Let exp be a closed expression. If exp reduces to a primitive value, val , using any of the three strategies of Sect. 11.2.3, then exp reduces to val following the by-name strategy. If exp diverges using the by-name strategy, then it also diverges under the other two strategies.*

Let us note that the theorem excludes the case in which a (closed) expression can yield a primitive value, val , in one strategy and yield *another* primitive value, val2 , using another strategy.⁵ Two strategies can therefore differ only by the fact that one determines a value while the other diverges, as we saw in our example. We cannot go into the details of how the theorem can be proved but it is important to stress that the following basic property is fundamental to its validity:

Once any strategy has been fixed in a given environment, the evaluation of all occurrences of a single expression always yields the same value

This property, which is immediately falsified when there are side effects, is taken by many authors as the *criterion* for a pure functional language: a language is purely functional if it satisfies this condition. This is a very important property which makes it possible to *reason* about a functional program and to which we will return in Sect. 11.5. Let us again note, before going on, that it is just by virtue of this property that lazy evaluation is correct. The value obtained by the evaluation of an expression does not change when another copy of the same expression is encountered (clearly, in the meanwhile, the environment has not been changed).

In the light of the preceding theorem, it can be asked what interest there is in the by-value strategy, given that by-name is the most general of all possible strategies. Moreover, reasons of efficiency would seem to suggest that we only adopt call-by-need strategies, given that it is not clear why useless redexes should be reduced, with the consequent waste of computing time. The point is that the efficiency case is not so simple. To implement a call-by-need strategy is, in general, more expensive than a simple call-by-value strategy. The by-value strategy has efficient implementations

⁵The theorem is no longer valid if the hypothesis that the value is *primitive* is removed. Having assumed that evaluation is not performed under abstraction, in fact, the two strategies can yield (functional) values that are distinct. By way of an example, define $I = \text{fn } x \Rightarrow x$ and $P = \text{fn } x \Rightarrow (\text{fn } y \Rightarrow y \ x)$. The term $P \ (I \ I)$ reduces to $\text{fn } y \Rightarrow y \ (I \ I)$ under the by-name strategy, while, using the by-value strategy, it reduces to $\text{fn } y \Rightarrow y \ I$. This difference is not very important in programming languages (where we mainly want to compute values of primitive type). In Sect. 11.6, we will define an abstract calculus in which reduction also occurs in the presence of abstractions. This calculus satisfies the *confluence* property (stronger and more general than Theorem 11.2), which we will discuss on page 361.

on conventional architectures, even if, sometimes, it performs unnecessary work (every time that an argument that is not required in the body of the function is evaluated, as happens with the second argument of the function *K* in Fig. 11.2). This last case, moreover, can be treated in an efficient manner with abstract evaluation strategies that attempt to identify useless arguments.

Among the functional languages that we have cited at the beginning of this chapter, LISP, Scheme and ML use a by-value strategy (also because they include major imperative aspects), while Miranda and Haskell (which are pure functional languages) use lazy evaluation.

11.3 Programming in a Functional Language

The mechanisms we described in the last section are sufficient to express programs for every computable function (they constitute, indeed, a Turing-complete language). This is the nucleus of every functional language, which, however, is too austere for us to be able to use as a real programming language. Every functional language, therefore, embed this nucleus in a wider context which provides mechanisms of different kinds, each aiming to make programming simpler and more expressive.

11.3.1 Local Environment

The mechanism of global definitions in the environment that we have used this far has too little structure for a modern language. As in conventional languages, it is appropriate to provide explicit mechanisms to introduce definitions with limited scope, as for example:

```
let x = exp in exp1 end
```

This introduces the binding of *x* to the value of *exp* in a scope which includes only *exp1*.

To tell the truth, the presence of functional expressions already introduces nested scopes and associated environments which are composed of the formal parameters of the function bound to the actual parameters. From the point of view of evaluation, we can indeed consider the construct

```
let x = exp in exp1 end
```

as syntactic sugar for

```
(fn x => exp1) exp.
```

The use of local scopes is too important, however, and justifies the introduction of special syntax for them.